

31 | 多线程：RN底层是如何保障交互体验的？

2022-12-1 蒋宏伟



你好，我是蒋宏伟。

在上一讲中，我们主要介绍的是 JavaScript 引擎，而 React Native 的一大特点，就是为执行 JavaScript 脚本单独开了一个异步线程。

和任何其他应用一样，React Native 应用也需要处理并行任务，比如，在用户滚动页面的时候，不仅要处理用户的滚动手势，并需要将页面绘制到新位置上，与此同时，还可能需要发起新的请求和做一些局部的渲染。只有单个线程时，处理任务就需要排队，前一个没完成，后一个就需要等待，这就容易导致交互的卡顿。

这一讲中，我们会围绕着 React Native 的多线程进行介绍，并会重点介绍新架构是如何改善交互体验的。

进程和线程

任何 React Native 应用，都是一个多线程的应用，这是由其架构设计的特点所决定的。

应用、进程、线程这些计算机的概念，关联性很强，也容易弄混。明确这些概念，有利于我们去理解 React Native 的多线程架构，因此我们借用[微软给出的关于应用、进程和线程的定](#)

义，来回忆一下这些熟悉的概念。

一个**应用**由一个或多个进程组成。一个**进程**，用最简单的话说，就是一个正在执行的程序，该程序的上下文中运行着一个或多个线程。一个**线程**是操作系统分配处理器时间的基本单位，一个线程可以执行进程代码的任何部分，包括目前正在被另一个线程执行的部分。

An application consists of one or more processes. A process, in the simplest terms, is an executing program. One or more threads run in the context of the process. A thread is the basic unit to which the operating system allocates processor time. A thread can execute any part of the process code, including parts currently being executed by another thread.

首先，**每个应用至少会有 1 个进程**。

为了更好地理解上面的定义，咱们可以使用 Mac 系统自带的 Activity Monitor 工具，去查看相关应用到底有哪些进程。比如，下图是我在电脑中开的 Chrome 应用和 zsh 命令行应用：

Process Name	% CPU	CPU Time	Threads
chrome_crashpad_handler	0.0	0.20	4
chrome_crashpad_handler	0.0	0.01	4
 Google Chrome	0.2	43:20.32	31
Google Chrome Helper	0.1	15:32.16	9
Google Chrome Helper	0.0	16.74	10
Google Chrome Helper	0.0	28.44	8
Google Chrome Helper	0.0	0.05	7
Google Chrome Helper (GPU)	0.2	44:16.04	15
Google Chrome Helper (Renderer)	0.1	5.68	17
Google Chrome Helper (Renderer)	0.0	0.07	10
Google Chrome Helper (Renderer)	0.0	4.13	16
zsh	0.0	0.23	1

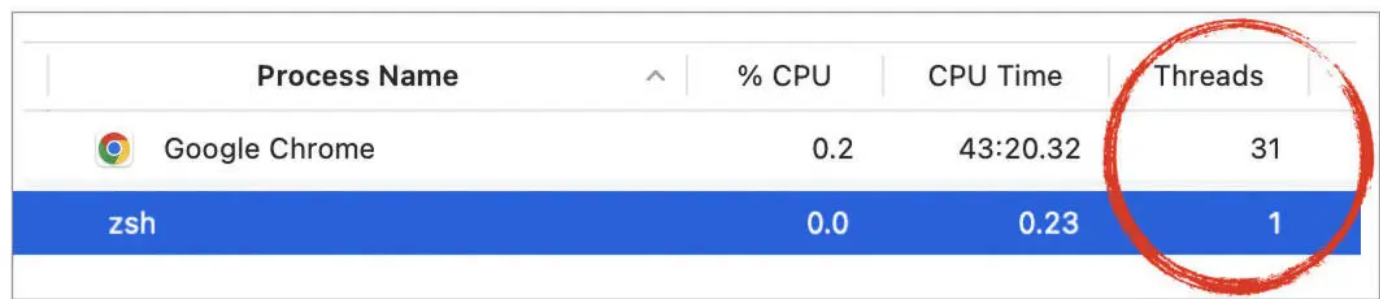
我开启的 Chrome 应用它有 10 多个进程。包括一个命名为 Google Chrome 的进程，若干个 crashpad_handler 进程和若干个 Helper 进程。这些 Helper 进程中，有的负责处理 GPU，

有的负责处理 Renderer。也就是说，一个应用可由多个进程组成。

而对于 zsh 这样的工具类应用，它非常简单，一个应用就只由 1 个进程组成。

其次，**每个进程至少会有1个线程**。

咱们将注意力放在 Google Chrome 进程和 zsh 进程上：



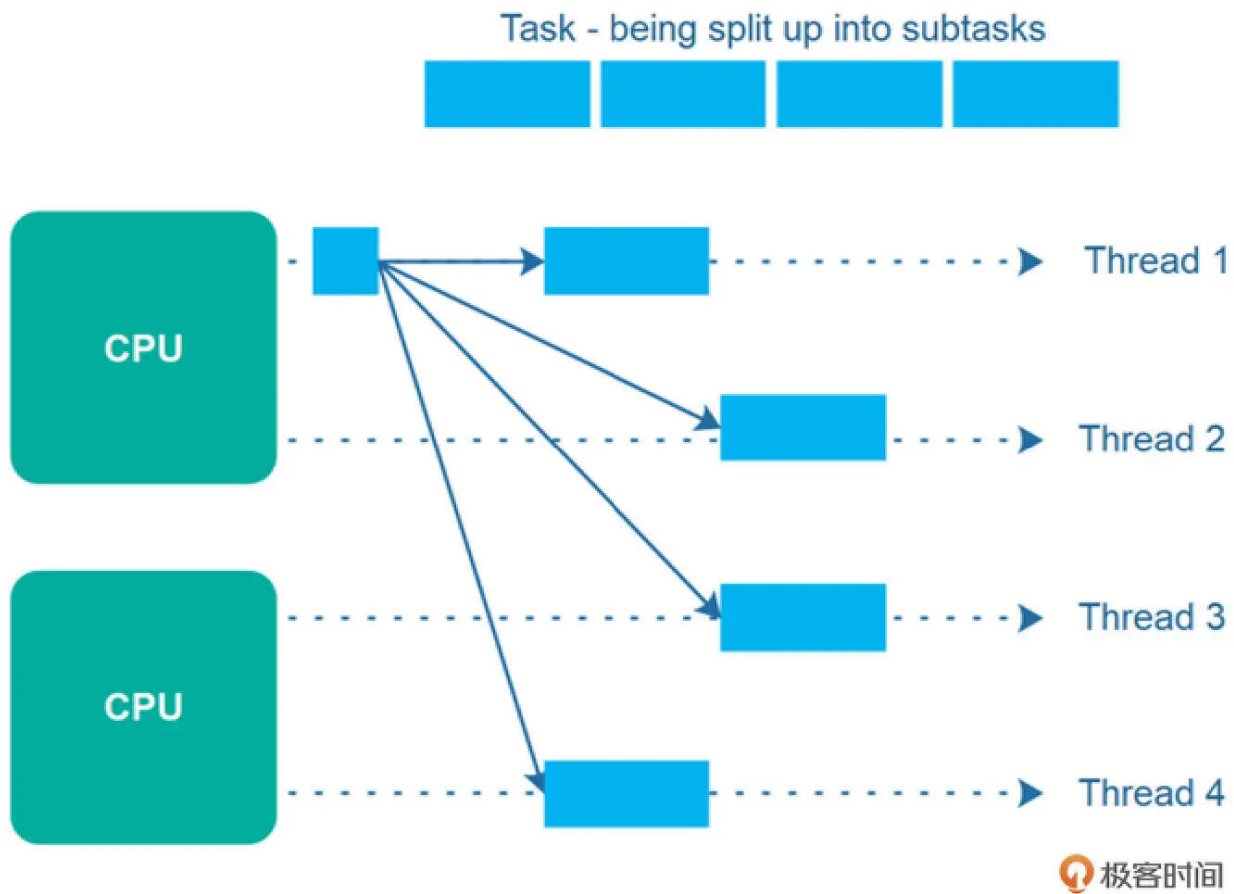
Process Name	% CPU	CPU Time	Threads
Google Chrome	0.2	43:20.32	31
zsh	0.0	0.23	1

Google Chrome 的进程它有 31 个线程，而 zsh 进程只有 1 个线程。

为什么 Chrome 浏览器它是多进程多线程的，而 zsh 命令行工具是单进程单线程的？因为二者要处理的任务的复杂度不同，特别是对并行任务的处理。

在计算机底层，线程是操作系统分配CPU时间的基本单元，同一个线程在同一个时间单元中，只能执行一个代码模块，这个代码模块代表的是某个具体的工作任务，比如计算1+1，在该线程执行1+1的时候它不能同时去执行其他任务，比如同时发起请求。

如果要想应用在计算的同时处理多个任务，那怎么办呢？**再另起一个线程**。在同一时间内，不同的线程可以同时处理不同的任务。一个多线程的示意图如下所示：



当我们给计算机分配一个任务时，如果这个任务可被分解成若干个**可独立执行子任务**，那么这时可以在主线程中创建若干个线程，来分别处理这些子任务。从计算机底层视角看，一个计算机可能会有多个 CPU，而每个 CPU 又是可以开多个线程的，因此计算机是可以同时处理这些可独立执行的子任务的，这就是并行的原理。

那什么时候我们需要多线程多进程来处理并发任务呢？

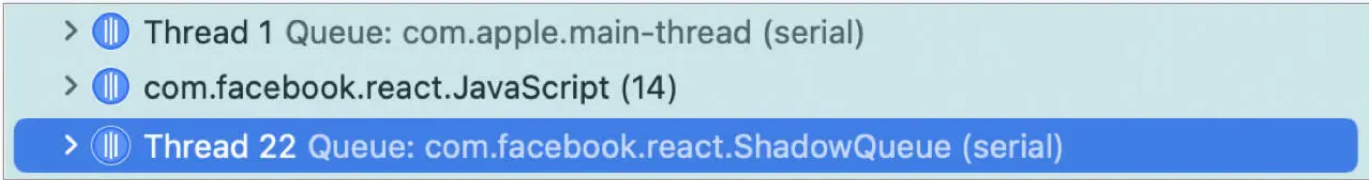
这需要根据任务的复杂度来确定。**任务的复杂度不一样，需要的进程和线程的数量也不一样**。打个比方，在一个大型项目中，要多个不同职能团队紧密合作，而且每个职能团队又会配备多个人，这就类似于多进程多线程。在一个中型项目中，一个多人团队就能完成好，这就类似单进程多线程模型。而一个人就能搞定的小型项目，就类似于单进程单线程。

React Native 中的多线程

而 React Native 框架底层，采用的是单进程多线程的设计。在 React Native 应用中至少会有三个线程：

1. 主线程（也叫做 UI 线程）
2. JavaScript 线程

3. Background 线程（也叫做 ShadowQueue 线程）



```
> Thread 1 Queue: com.apple.main-thread (serial)
> com.facebook.react.JavaScript (14)
> Thread 22 Queue: com.facebook.react.ShadowQueue (serial)
```

主线程是唯一一个可以操作视图的线程。React Native 应用本质是 iOS 应用或 Android 应用，因此，在启动时，操作系统也会为其创建唯一可操作视图的主线程。无论是 iOS 应用还是 Android 应用，在启动应用的时候，系统都会为该应用创建一个主线程，来负责用户交互和界面绘制，而除了主线程外，其他线程是不可以操作视图的。

JavaScript 线程是用于执行 JavaScript 脚本的线程。大多数情况下，JavaScript 脚本会交由 JavaScript 引擎来执行，比如处理点击手势、拼接接口字段、发起请求和执行 React Render 函数等任务。但值得注意的是，并不是任何的 JavaScript 脚本的执行都在 JavaScript 线程进行。

从源码中，可以看到 JavaScript 线程的名字叫做 `com.facebook.react.JavaScript`。

```
static NSString *const RCTJSThreadName = @"com.facebook.react.JavaScript";
```

ShadowQueue 线程是用于布局的线程。React Native 应用的布局任务，会交由 Yoga 布局引擎来执行，而 Yoga 布局引擎的执行，大多数情况下，是在 ShadowQueue 线程执行的。

从源码中，可以看到 ShadowQueue 线程的名字叫做 `com.facebook.react.ShadowQueue`。

```
char *const RCTUIManagerQueueName = "com.facebook.react.ShadowQueue";
```

除了以上三大线程之外，React Native 还会开启另外一些线程用于任务的执行。在 React Native iOS 源码中，你可以找到下面这些单独创建的线程：

```
dispatch_queue_create("com.facebook.react.ModuleClassesSyncQueue", DISPATCH_QUEUE_CONCURRENT);

dispatch_queue_create("com.facebook.react.WebSocketExecutor", DISPATCH_QUEUE_SERIAL);

dispatch_queue_create("com.facebook.react.AsyncLocalStorageQueue", DISPATCH_QUEUE_SERIAL);
```

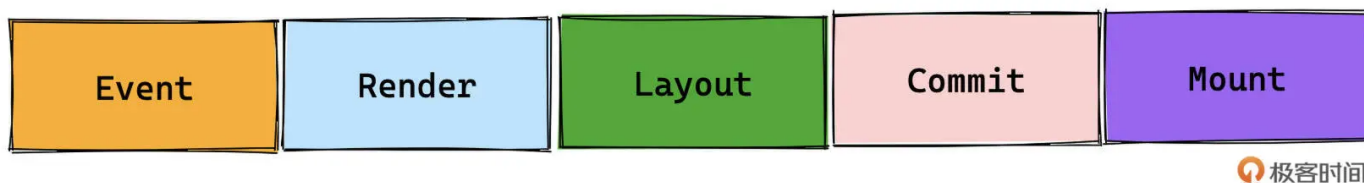
以上这些能够单独执行的三类任务，包括 Module 模块、WebSocket 长链接和 Storage 存储，每次调用相关模块的时候，React Native 都会为它们创建单独的线程。当然，还有 HTTP 请求之类的任务，也都有单独的线程进行处理，就不一一列举了。

简而言之，一个 React Native 应用有三个核心线程，以及根据任务情况随时创建的独立线程。

渲染与线程

对于任何的 GUI 应用而言，保障渲染体验是其重中之重。而多线程机制正是保障 React Native 渲染体验的核心要素之一。

一般而言，在 React Native 中，一个渲染任务可分为5个子任务。比如，先处理点击手势（Event），接着执行 React Render 函数，之后 Yoga 引擎开始计算布局（Layout），再然后框架提交（Commit）为渲染而准备的“React Shadow Tree”，最后将挂载（Mount）“React Shadow Tree”，并在屏幕上显示。示意图如下：



从上述流程中，我们可以发现，渲染子任务之间的关系必须是串行的，它的后一个子任务必须等待前一个子任务执行完成后才能继续执行。

在介绍多线程时，我们提到过，只有当任务可被分解成若干个可独立执行子任务时，多线程才能并行地处理它们，从而提高任务整体执行的效率。虽然渲染任务能被分解成若干个子任务，但是这些子任务之间是有先后顺序和依赖关系的，并不能完全独立地执行，这就不满足多线程并行执行的要求。

那么，为什么 React Native 还要弄三个线程来处理渲染任务？要弄清楚这点，就必须得弄清楚 React Native 的线程模型了。

React Native 新架构的线程模型非常复杂，并不是简单的在三个核心线程之间异步地执行，而是根据不同场景采用不同的方案来执行渲染任务，[官方文档](#)中给出了 6 种场景下的 6 种渲染任务的执行方案。具体的方案，就不一一列举了，咱们只需握住核心：**不同优先级的渲染任务用不一样的线程解决方案。**

其中，最重要的两个场景方案是：

1. 在主线程中渲染：对于高优先级的渲染任务，它的所有子任务会同步地的主线程上执行。
2. 在背景线程中渲染：对于一般场景的渲染任务，它的大分子任务会在 JavaScript 线程和 Background 线程中执行。

Render in the UI Thread: When there is a high priority event on the UI Thread, the renderer is able to execute all the render pipeline synchronously on the UI thread.

Render in a Background Thread: This is the most common scenario where most of the render pipeline happens on JavaScript and background thread.

在主线程中渲染

由于 iOS 和 Android 的更新 UI 机制的限制，非主线程一般是不能操作 UI 的，加之多线程本身也会增加复杂度，因此，只在主线程中渲染的流程最简单，因此，我们先讨论它。

有些同学可能会怀疑，只有主线程，会不会导致 React Native 的卡顿。这种担心是不无道理的，因为，对于纯 Native 应用而言，它可以直接操作 UI，而 React Native 应用还需要额外考虑 JavaScript 执行的时间，这就更可能会导致卡顿。

但可能会卡顿并不等同于一定会卡顿。有些高优先级的渲染任务，就非常适合只在主线程中执行。根据渲染原理可知，若在1s之内能够稳定地渲染出60帧，也就是16ms每帧，那么渲染就是流畅的；只有超过16ms，用户才可能会觉得卡顿。换句话说，在单次渲染任务中，只要 JavaScript 执行和 UI 操作的总耗时不超过 16ms，那么，用户就不会感觉卡顿。

例如，在 [《07 | TextInput：如何实现一个体验好的输入框？》](#) 中提到的 `TextInput`，它就有个“隐藏”事件，可以触发这种高优先级的同步渲染任务。

```
<TextInput
  unstable_onChangeSync={event => {}}
/>
```

此时，用户输入文字触发的回调方法，以及后续的其他渲染子任务，都会在主线程中执行。在主线程中渲染的示意图如下：



主线程执行的 5 个任务依次是，先执行关于文字输入事件（Event）的回调函数，若回调函数触发了 `setState` 方法后，就会接着执行 React 的“Render”方法，然后在计算具体的元素布局（Layout），再然后框架提交（Commit）为渲染而准备的“React Shadow Tree”，最后将挂载（Mount）“React Shadow Tree”，并在屏幕上显示。

当然，官方也提示了这种高优先级的同步渲染的风险性：`unstable_onChangeSync` 回调将会在主线程中执行，并且它可能会导致掉帧。

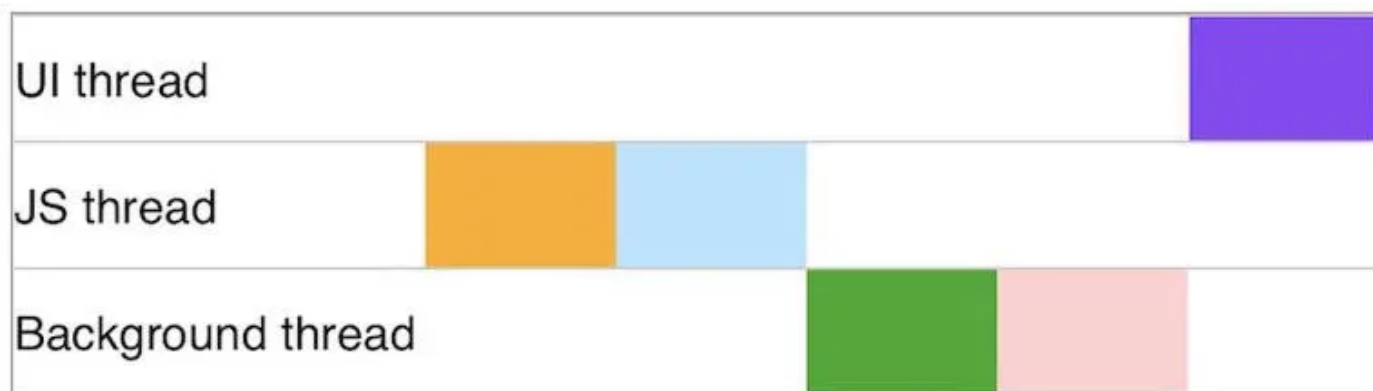
Callback will be called on the main thread and may result in dropped frames.

在背景线程中渲染

然而，正如我们前面提到的，React Native 应用相对于 Native 应用而言，多了 JavaScript 执行这一步，这就增加了渲染任务不能在16ms内执行完成的风险。因此，在大部分情况下，开发者只需使用 `TextInput` 组件的 `onChange` 回调来更新页面即可。

```
<TextInput
  onChange={event => {}}
/>
```

和其他组件方法一样，TextInput 组件的 onChange 方法是多线程异步执行的，这种机制就叫做**在背景线程中渲染**。其示意图如下：



该渲染任务依旧可分为 5 个子任务，Event、Render、Layout、Commit 和 Mount，并且这些子任务也需要依次执行。但和只在主线中渲染的方案不同，该方案极大地降低了主线程卡顿的问题，因为 Event 和 Render 相关的 JavaScript 代码是在 JavaScript 线程中执行的，而 Layout、Commit 是在背景线程中执行的。

由于主线程、JavaScript 线程和 Background 线程是三个相互独立的线程，因此它们可以并行地执行任务，并且不相互干扰。例如，在执行前四个子任务（Event、Render、Layout、Commit）时，主线程是空闲的状态，这时，主线程可以并行地处理类似于用户滚动之类的交互任务。这就意味着，即便前四个子任务中的耗时超过了 16ms，但只要主线程的 Mount 子任务不超过 16ms，用户也感觉不到卡顿。这就是，在背景线程中渲染方法的优势。

总结

总而言之，React Native 新架构在底层会创建至少三个线程。框架默认采用三线程异步渲染，对于高优先级的渲染任务可采用同步渲染。

三线程异步渲染的优势在于，即便 JavaScript 任务执行时间过长，但主线程依旧可以同时响应用户的操作。例如，极端情况下 JavaScript 假死，页面依旧可以滑动。根据实践经验，大部分的卡顿确实是 JavaScript 中的任务执行耗时长导致的。

此外，某些特定场景下，提高交互事件优先级，使用主线程同步渲染，也是一种保障交互体验的手段。例如，在用户输入 `TextInput` 文字，并有大范围渲染页面的场景下，提高用户输入文字的优先级，可减少渲染导致的输入文字卡顿问题。

思考题

请你想想，React Native 是否可以设计成只使用 JavaScript 线程和主线程的“双线程”异步渲染模型呢？三线程异步渲染，相对于“双线程”的优势在哪？

欢迎在留言区分享你的经验、交流学习心得或者提出问题，如果觉得有收获，也期待你把今天的内容分享给更多的朋友！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 30 | JavaScript 引擎：双向通讯底层原理是什么？

下一篇 32 | Fabric：新渲染器的演进之路

精选留言 1